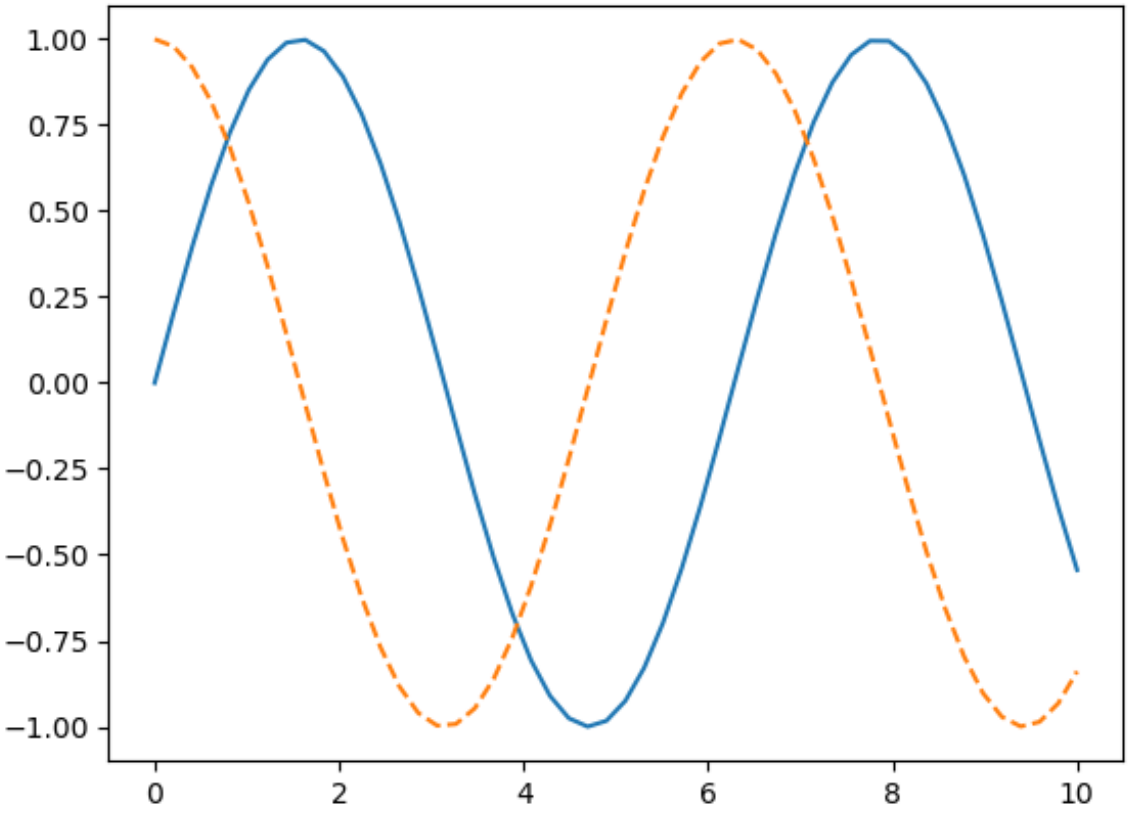
In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

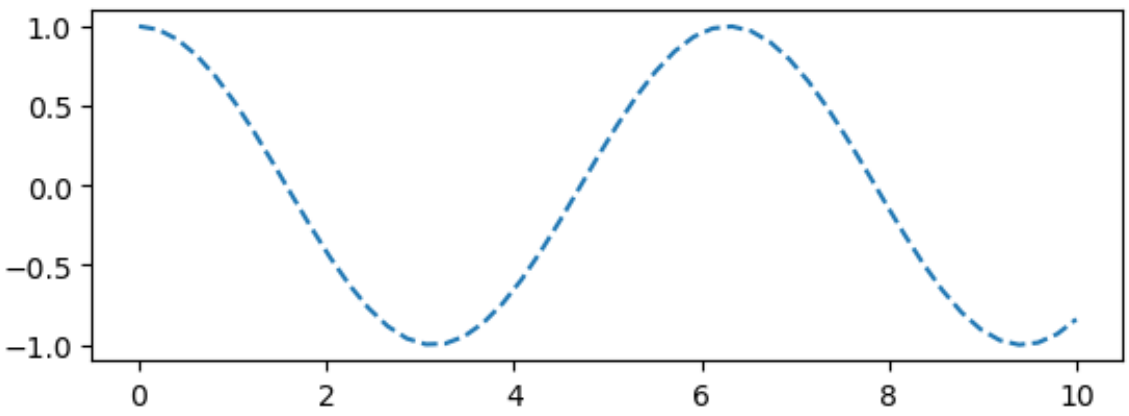
In [2]:
```
%matplotlib inline
x1 = np.linspace(0,10,50)
plt.plot(x1,np.sin(x1),ls='-')
plt.plot(x1,np.cos(x1),ls='--')
```

Out[2]:
```
[<matplotlib.lines.Line2D at 0x152973110>]
```



In [3]:
```
plt.subplot(2,1,1)
plt.plot(x1,np.cos(x1),ls='--')
```

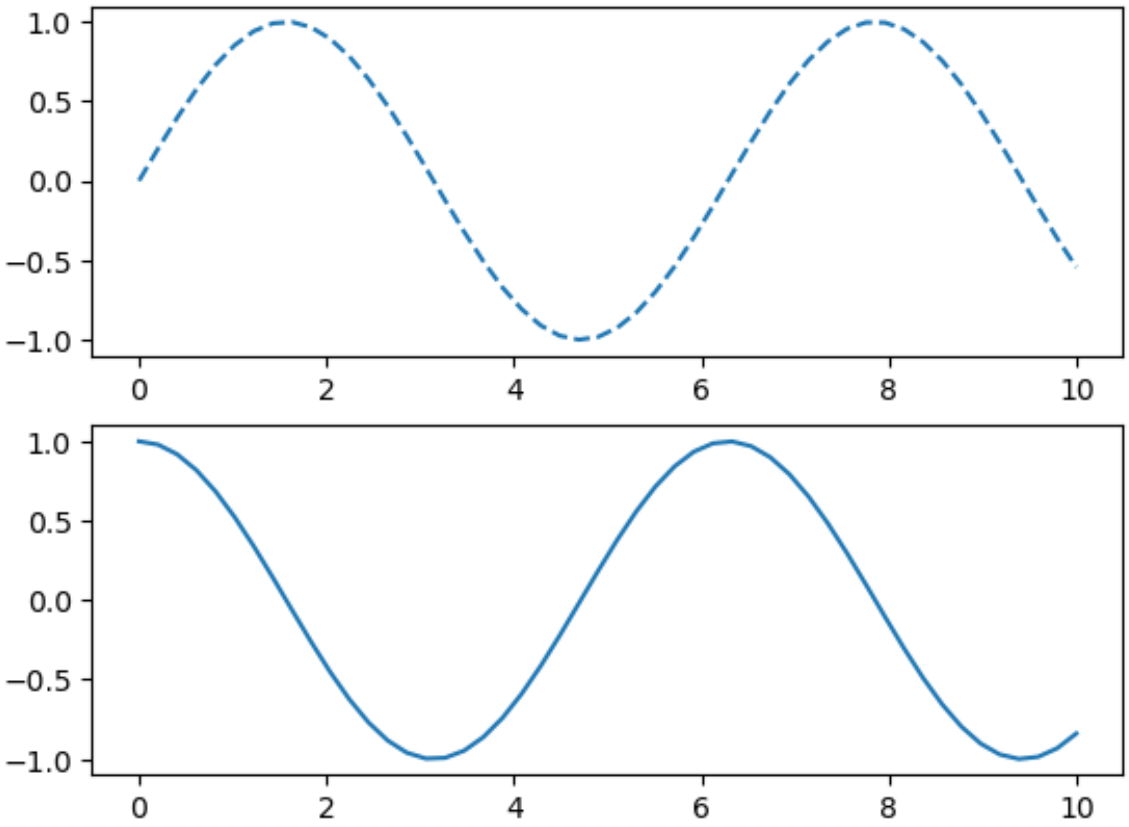
Out[3]:
```
[<matplotlib.lines.Line2D at 0x152c107d0>]
```



In [4]:
```
plt.figure()
```

```
plt.subplot(2,1,1)
plt.plot(x1,np.sin(x1),'--')

plt.subplot(2,1,2)
plt.plot(x1,np.cos(x1),'-')
```

Out[4]: [<matplotlib.lines.Line2D at 0x152cfd590>]



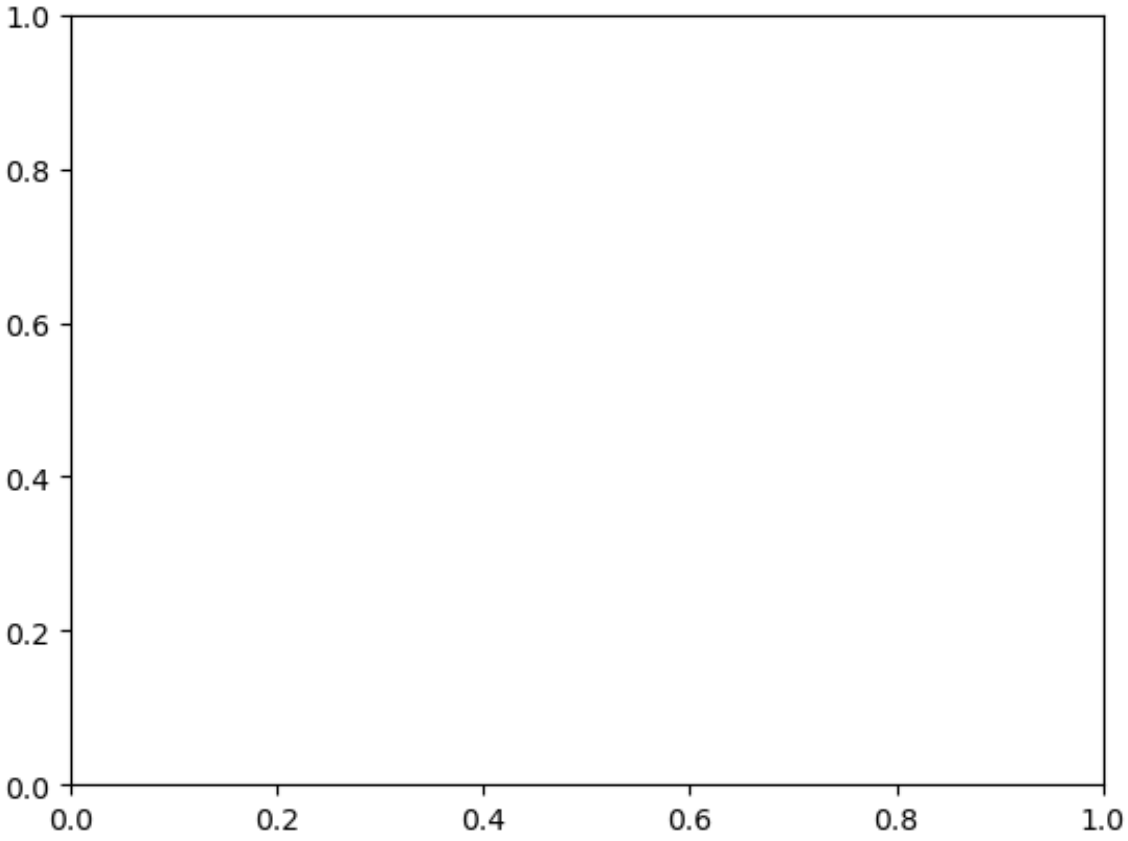
In [5]:
```
plt.gcf()
```

Out[5]: <Figure size 640x480 with 0 Axes>

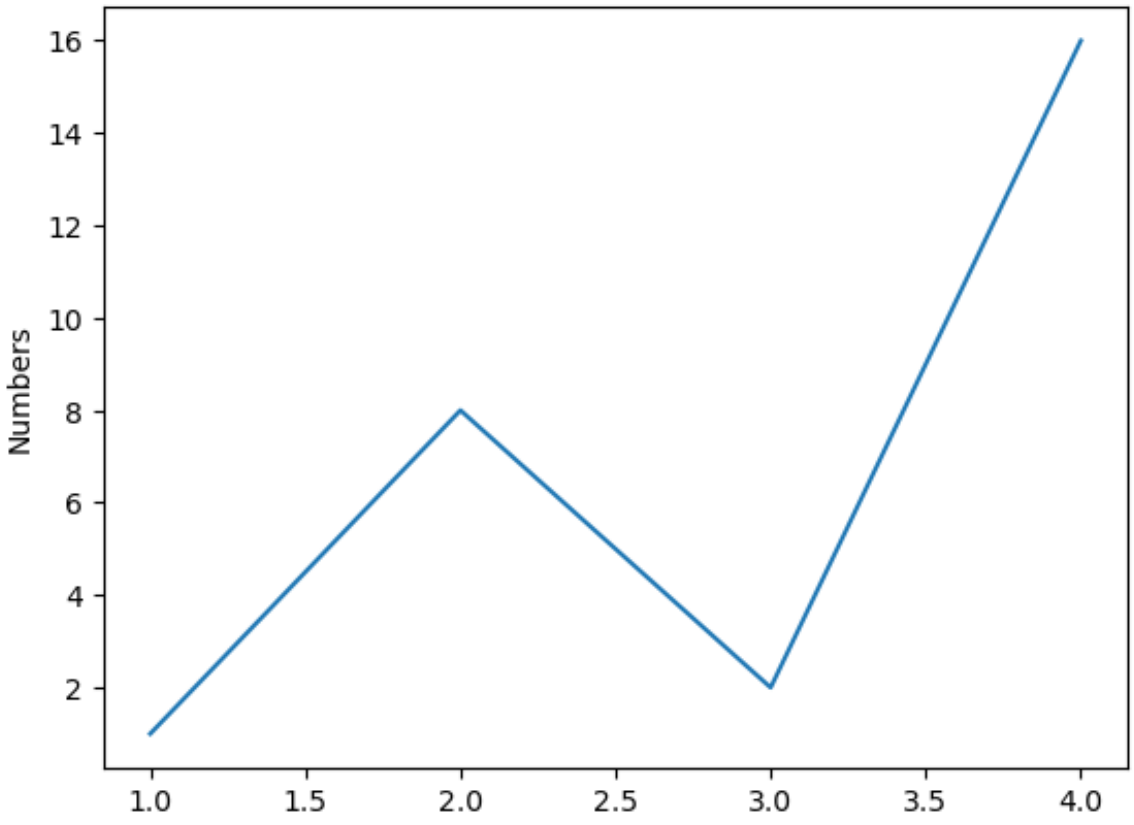
<Figure size 640x480 with 0 Axes>

In [6]:
```
plt.gca()
```

Out[6]: <Axes: >



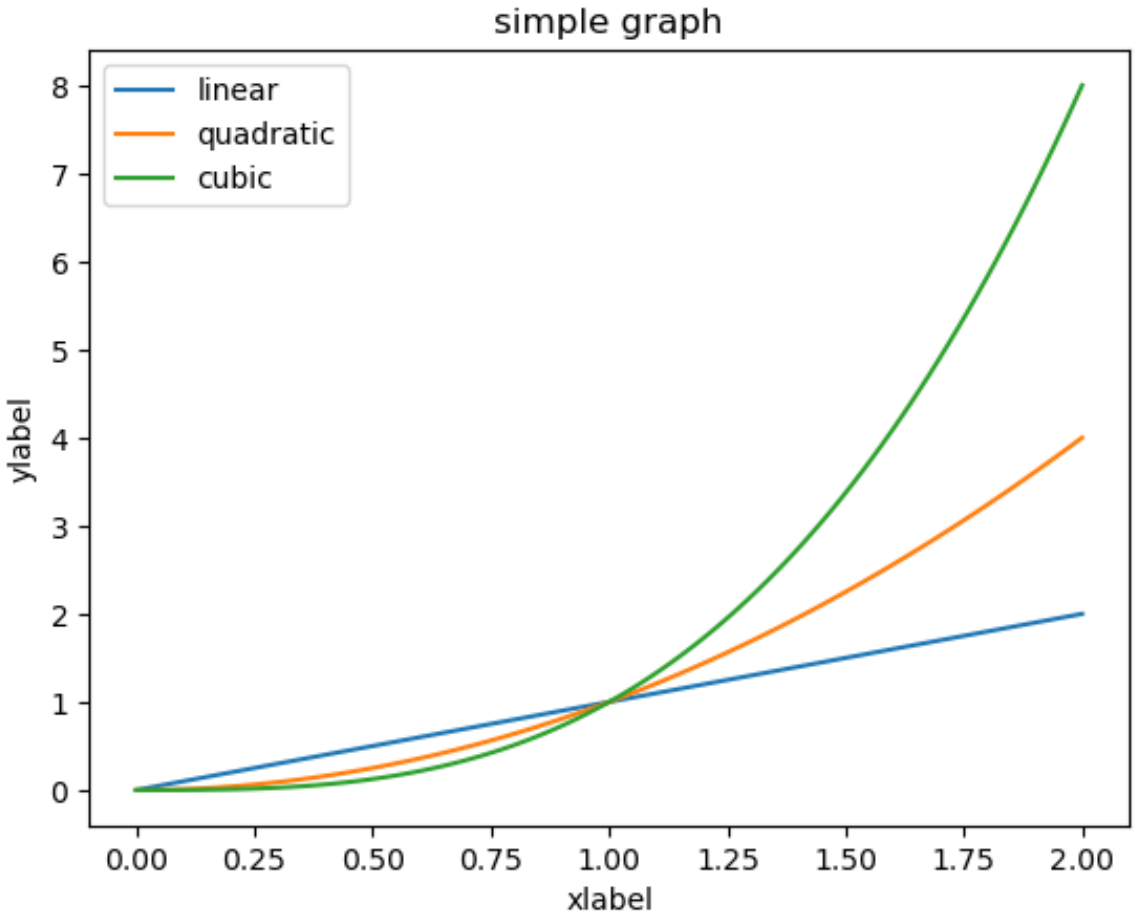
In [7]:
```
plt.plot([1,2,3,4], [1,8,2,16])
plt.ylabel('Numbers')
plt.show()
```



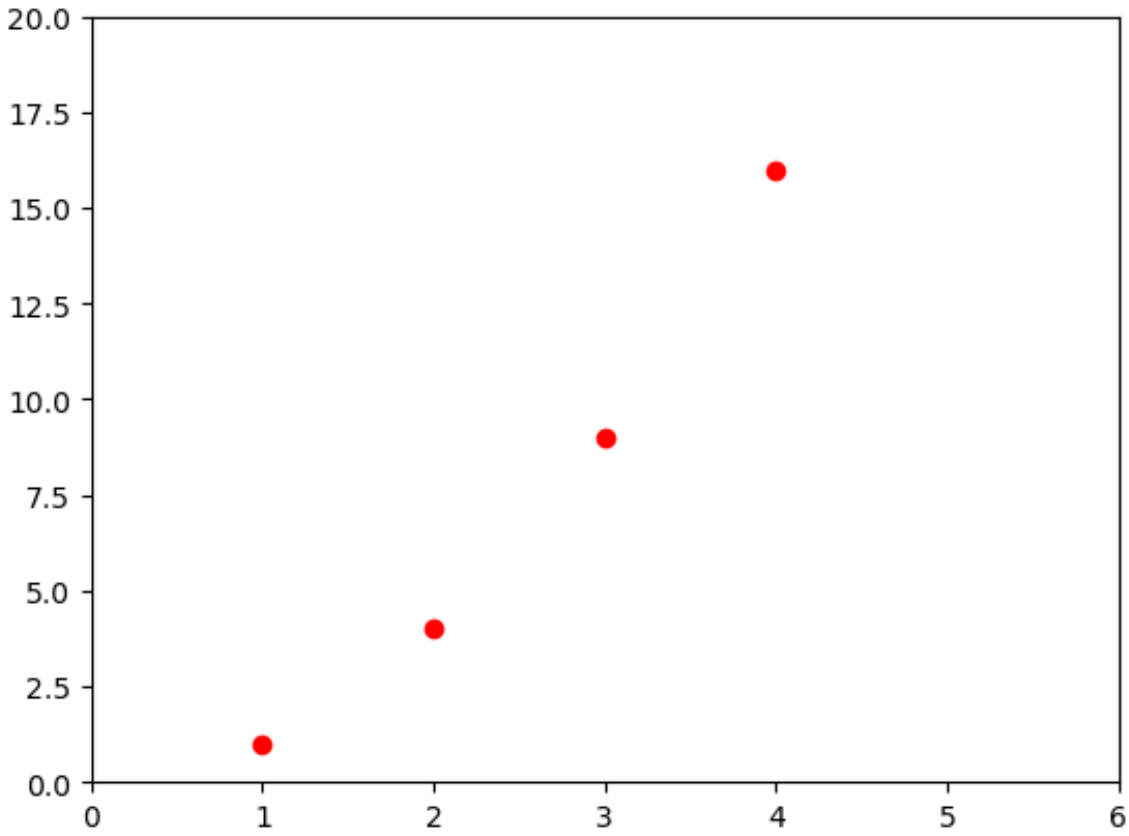
In [8]:
```
x = np.linspace(0,2,100)

plt.plot(x,x,label='linear')
```

```
plt.plot(x,x**2,label='quadratic')
plt.plot(x,x**3,label='cubic')

plt.ylabel('ylabel')
plt.xlabel('xlabel')

plt.title('simple graph')
plt.legend()
```

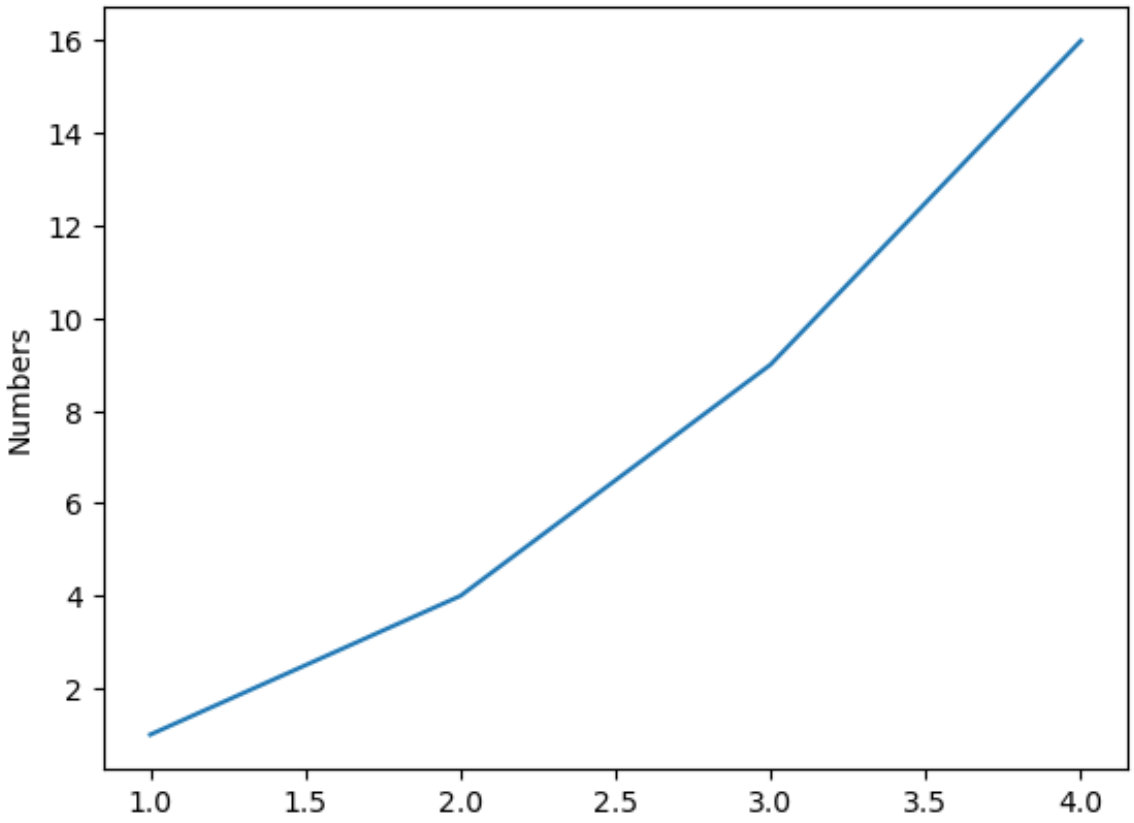
Out[8]: <matplotlib.legend.Legend at 0x1529278c0>



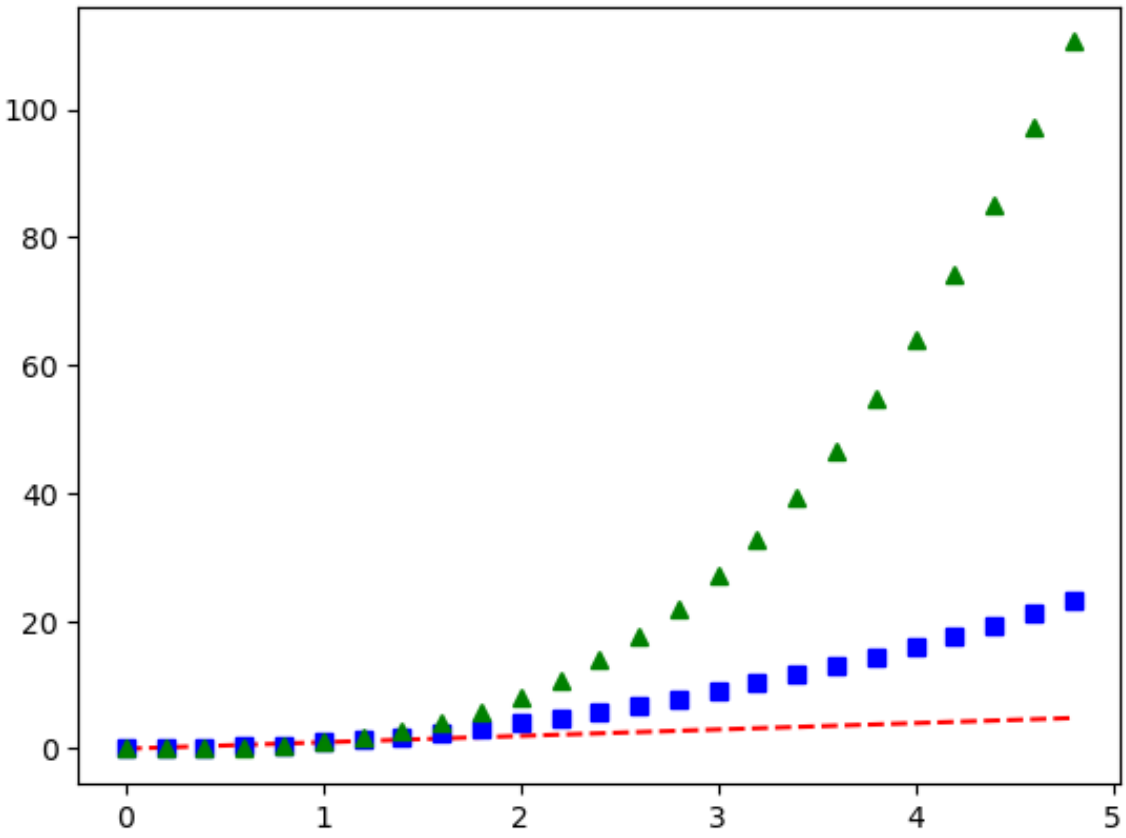
In [9]:
```
plt.plot([1, 2, 3, 4], [1, 4, 9, 16], 'ro')
plt.axis([0, 6, 0, 20])
plt.show()
```



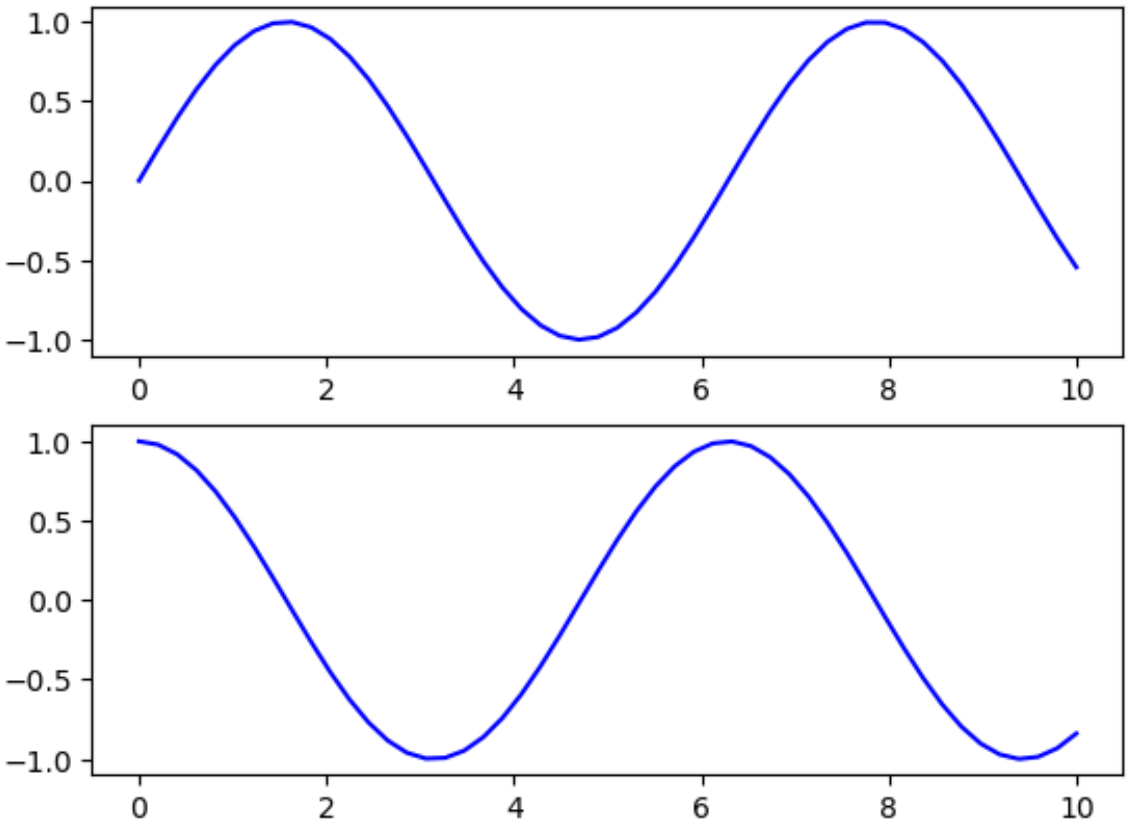
In [11]:
```
plt.plot([1,2,3,4],[1,4,9,16])
plt.ylabel('Numbers')
plt.show()
```



In [13]:
```
t = np.arange(0.,5.,0.2)
plt.plot(t,t,'r--',t,t**2,'bs',t,t**3,'g^')
plt.show()
```

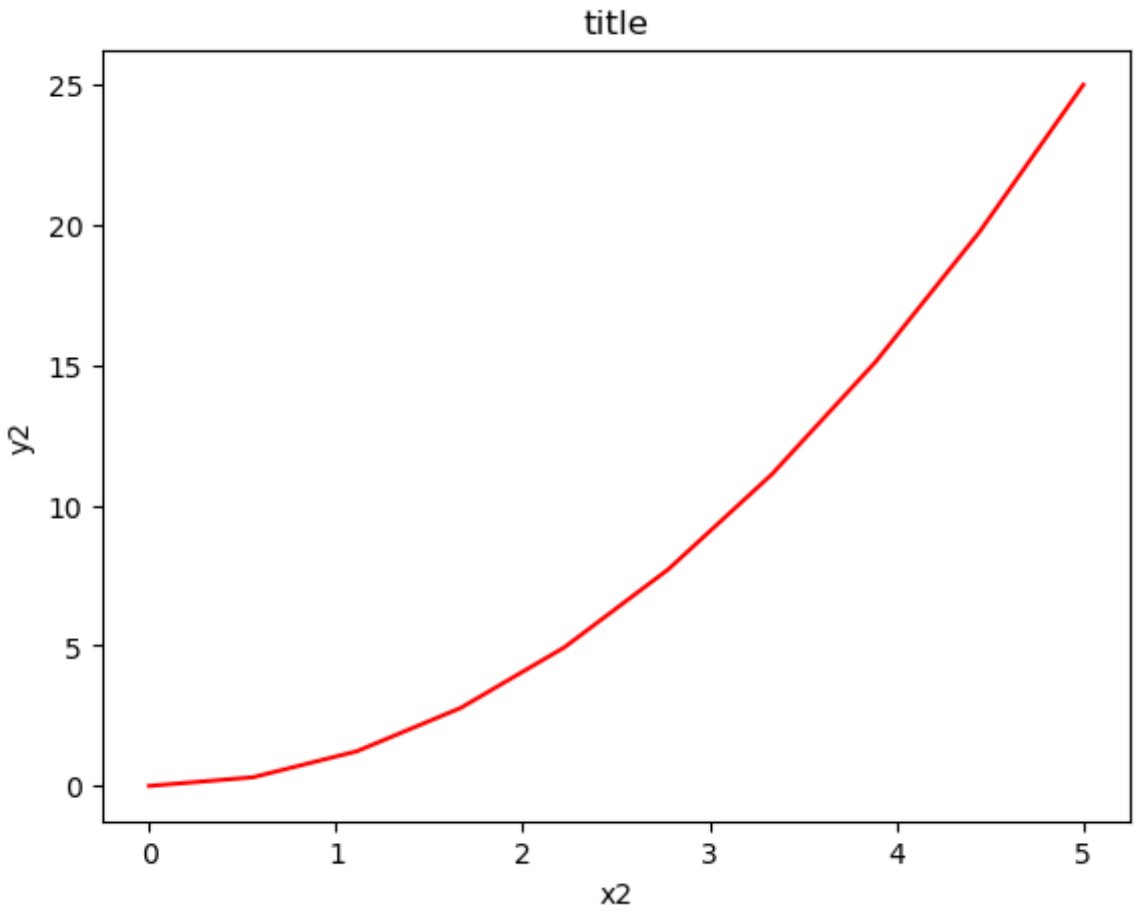


In [17]:
```
fig, ax = plt.subplots(2)

ax[0].plot(x1, np.sin(x1), 'b-')
ax[1].plot(x1, np.cos(x1), 'b-');
```

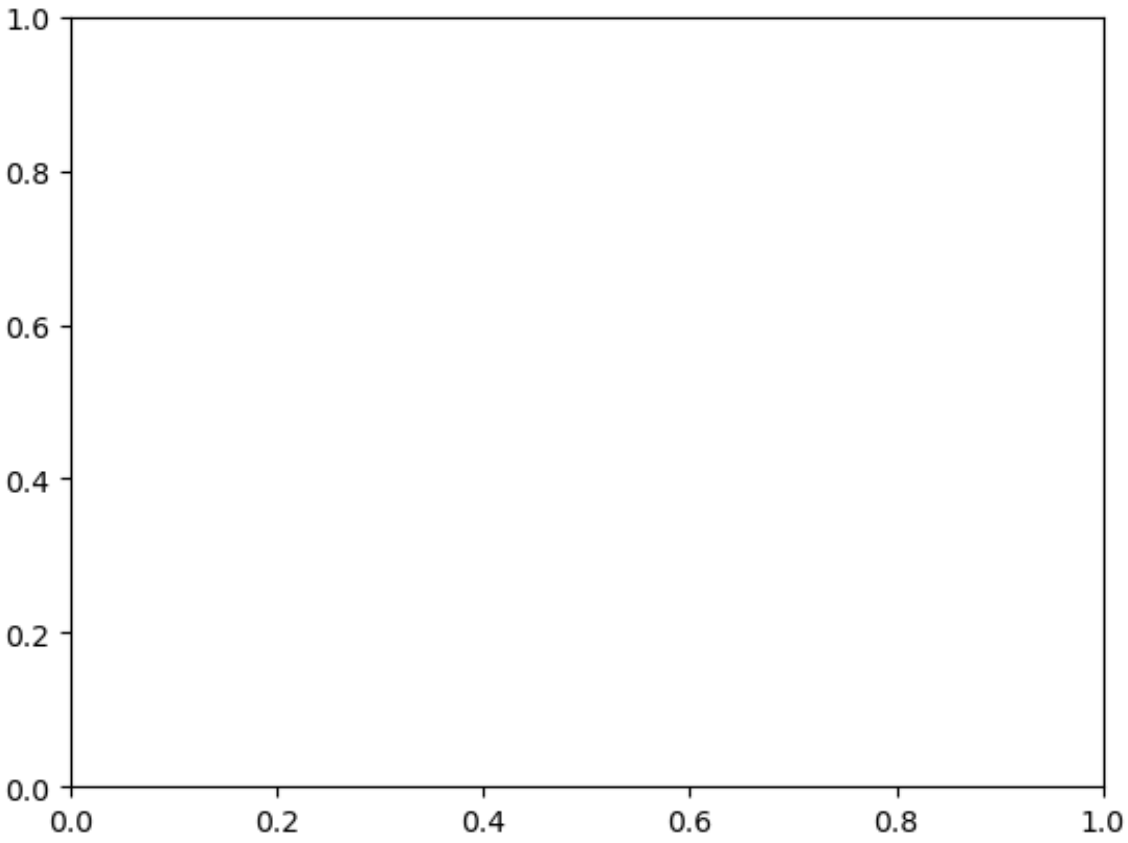


In [18]:
```
fig = plt.figure()
```

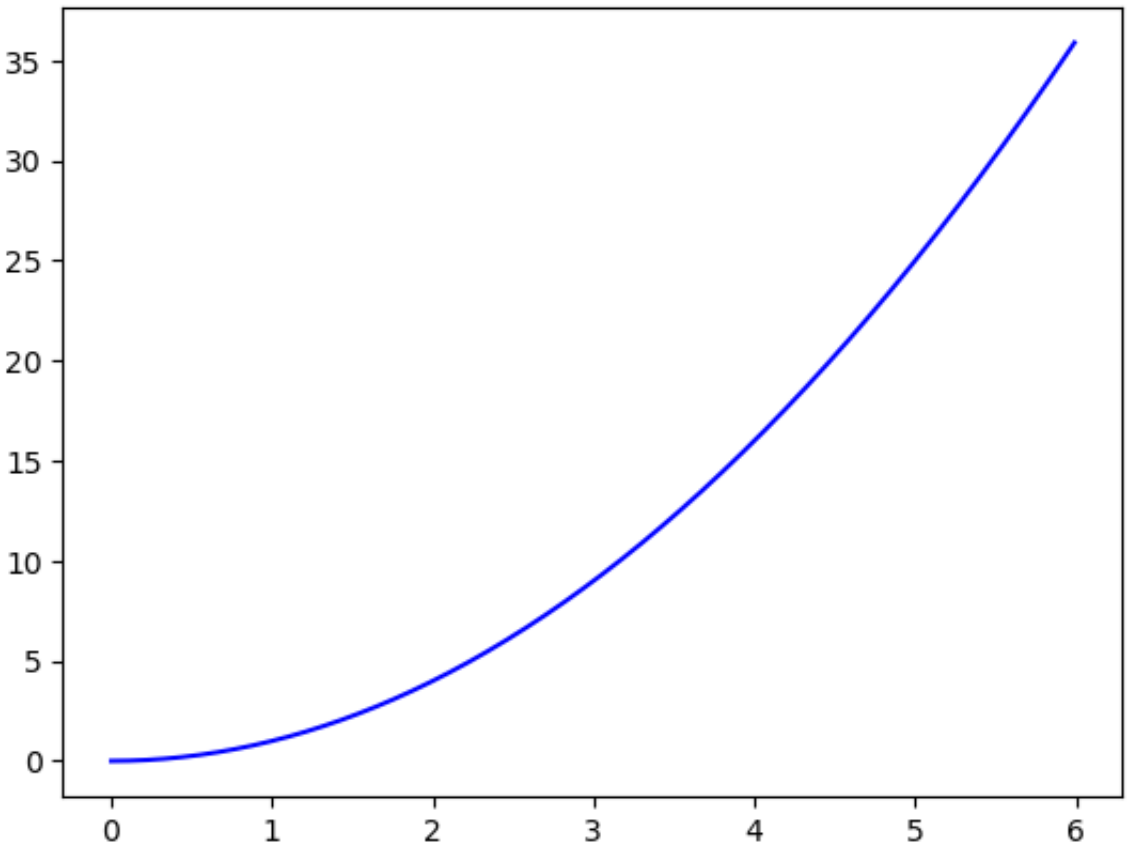
```
x2 = np.linspace(0, 5, 10)
y2 = x2 ** 2

axes = fig.add_axes([0.1, 0.1, 0.8, 0.8])

axes.plot(x2, y2, 'r')

axes.set_xlabel('x2')
axes.set_ylabel('y2')
axes.set_title('title');
```



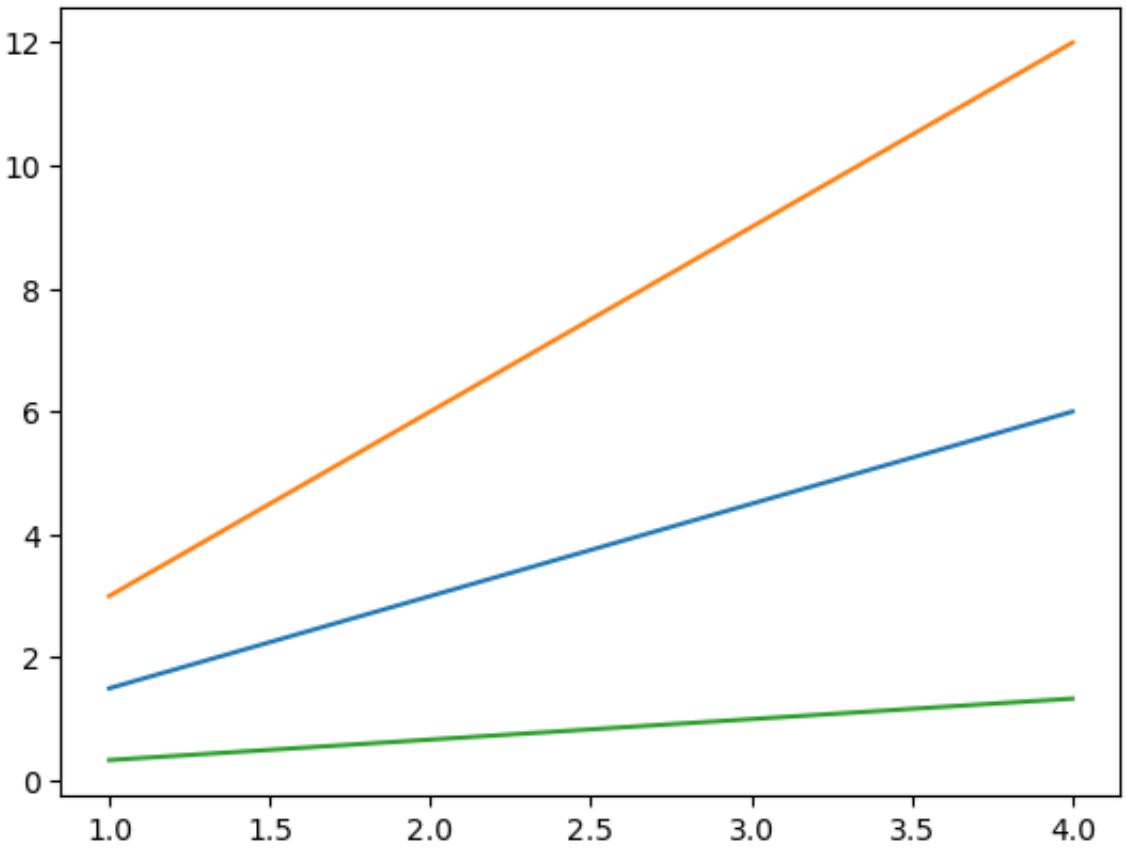
In [19]:
```
fig = plt.figure()
ax = plt.axes()
```



In [20]:
```
x3 = np.arange(0.0, 6.0, 0.01)

plt.plot(x3, [xi**2 for xi in x3], 'b-')

plt.show()
```



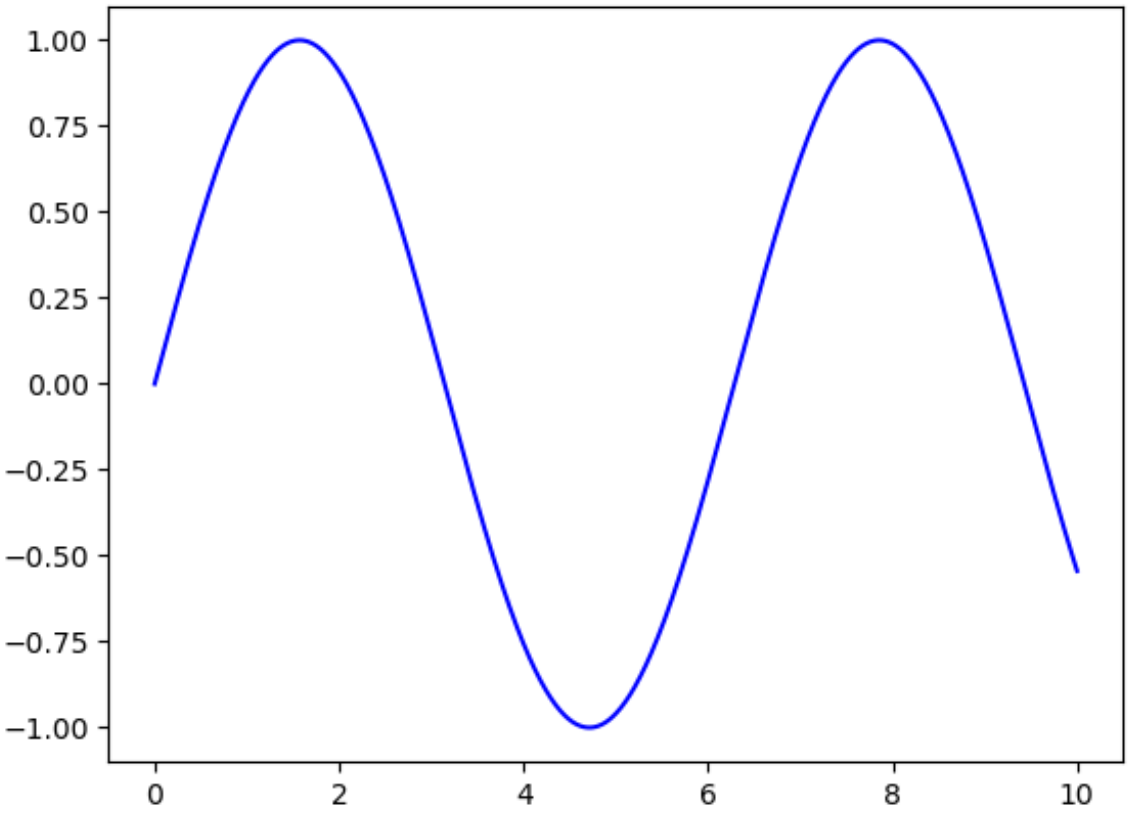
In [21]:
```
x4 = range(1, 5)

plt.plot(x4, [xi*1.5 for xi in x4])

plt.plot(x4, [xi*3 for xi in x4])

plt.plot(x4, [xi/3.0 for xi in x4])

plt.show()
```



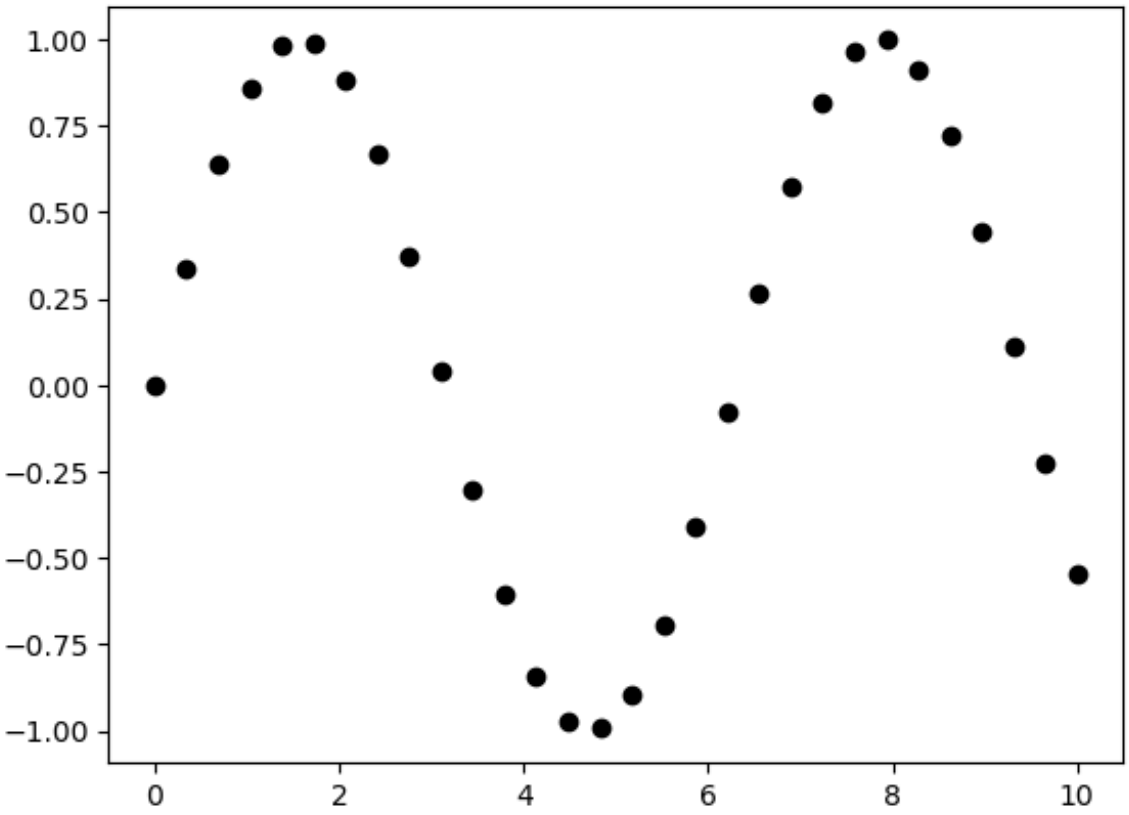
In [22]:
```
fig.canvas.get_supported_filetypes()
```

Out[22]:
```
{'eps': 'Encapsulated Postscript',
 'jpg': 'Joint Photographic Experts Group',
 'jpeg': 'Joint Photographic Experts Group',
 'pdf': 'Portable Document Format',
 'pgf': 'PGF code for LaTeX',
 'png': 'Portable Network Graphics',
 'ps': 'Postscript',
 'raw': 'Raw RGBA bitmap',
 'rgba': 'Raw RGBA bitmap',
 'svg': 'Scalable Vector Graphics',
 'svgz': 'Scalable Vector Graphics',
 'tif': 'Tagged Image File Format',
 'tiff': 'Tagged Image File Format',
 'webp': 'WebP Image Format'}
```

In [23]:
```
fig = plt.figure()
ax = plt.axes()
x5 = np.linspace(0, 10, 1000)
ax.plot(x5, np.sin(x5), 'b-');
```

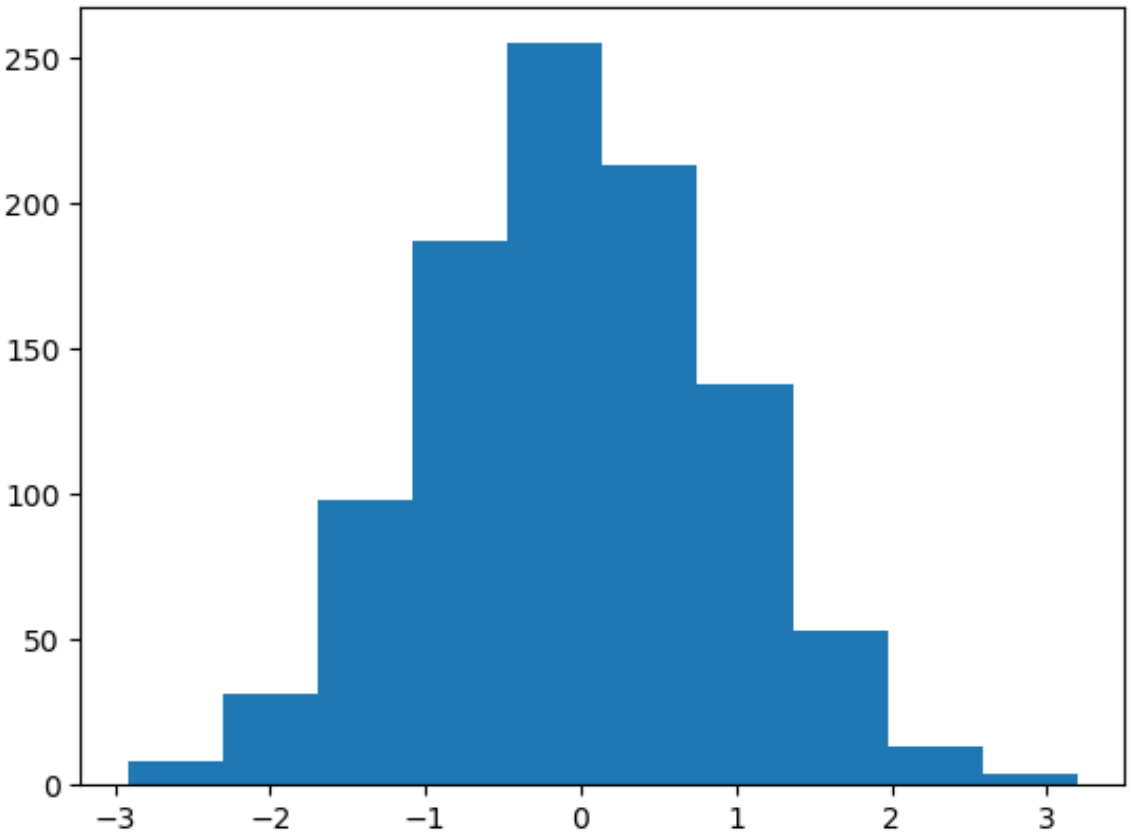


In [24]:
```
x7 = np.linspace(0, 10, 30)

y7 = np.sin(x7)

plt.plot(x7, y7, 'o', color = 'black');
```

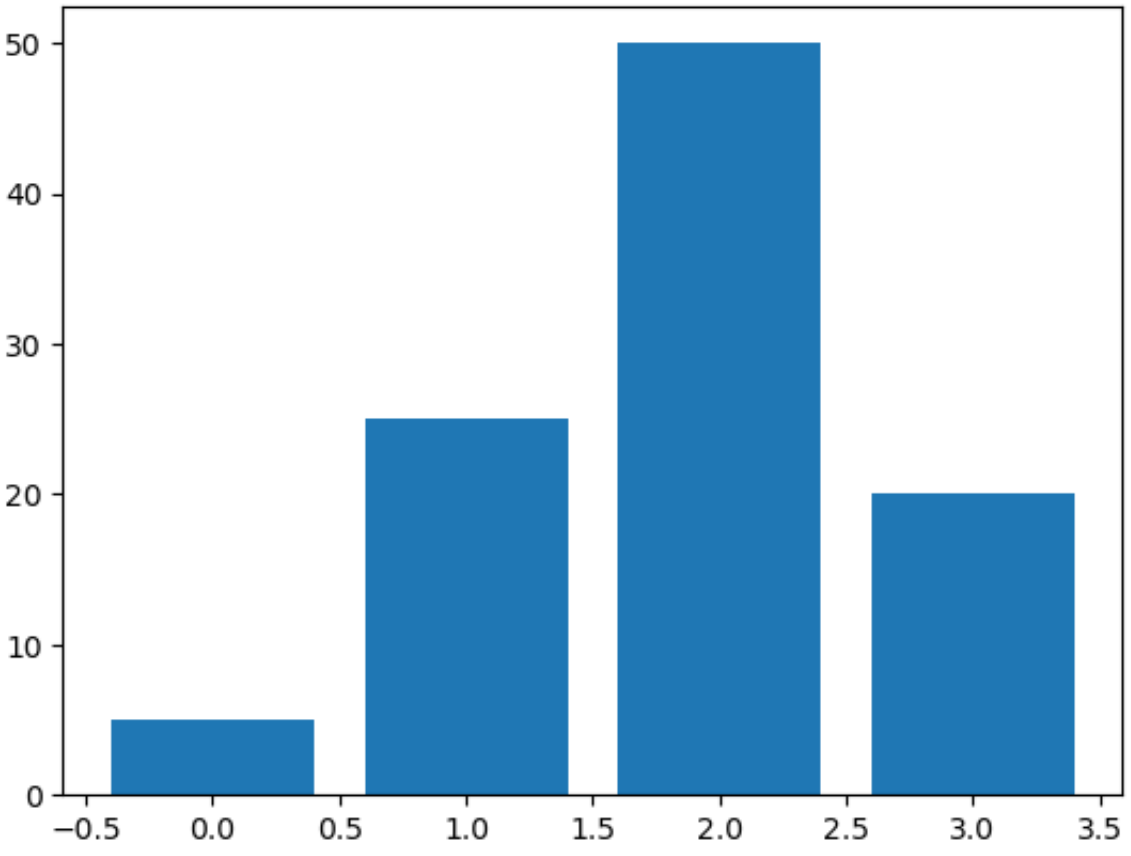


In [26]:
```
data1 = np.random.randn(1000)
```

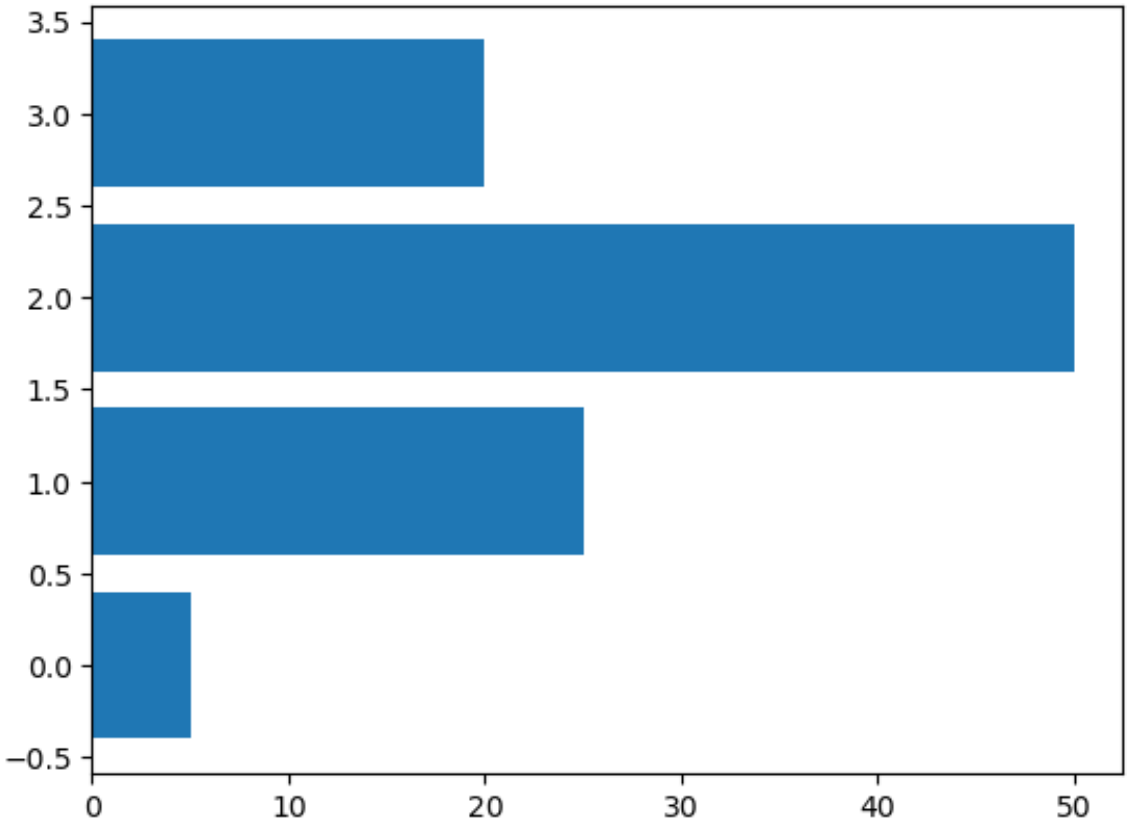
```
plt.hist(data1)
plt.show()
```



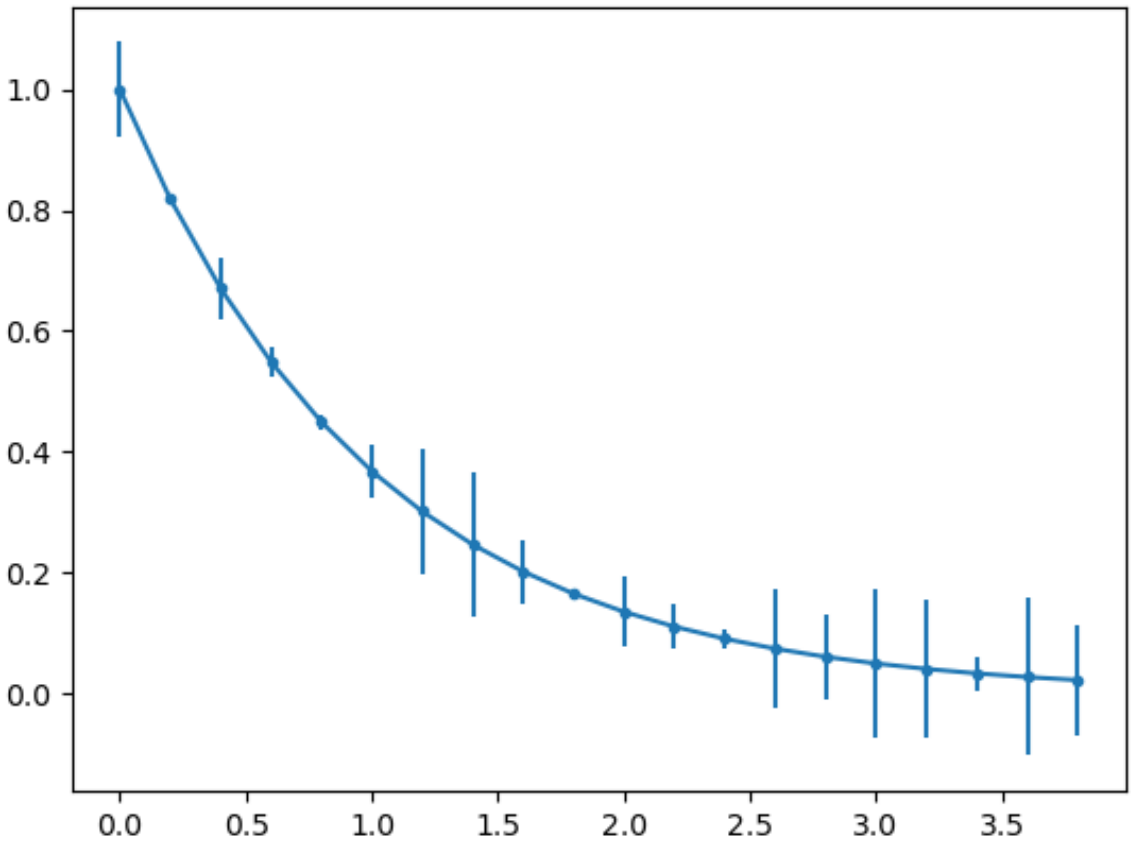
In [30]:
```
data2 = [5. , 25. , 50. , 20.]
plt.bar(range(len(data2)), data2)
plt.show()
```



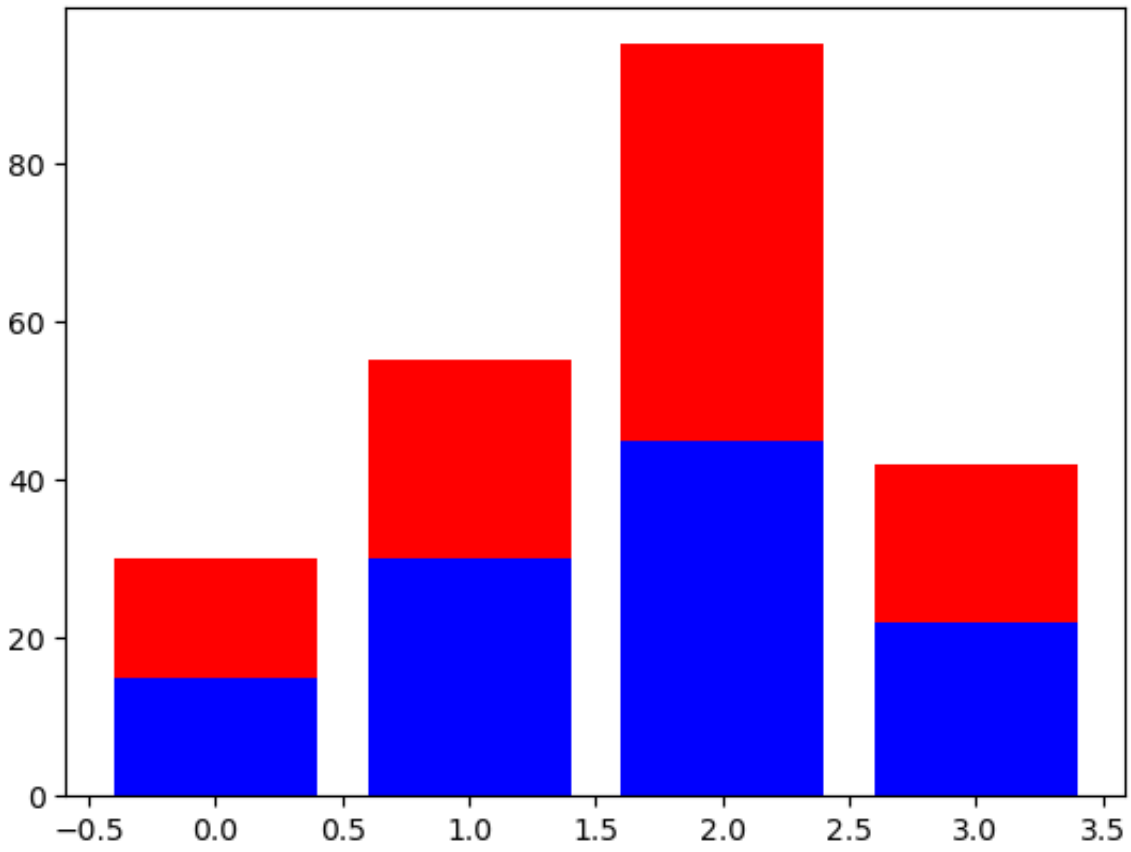
In [31]:
```
plt.barh(range(len(data2)),data2)
plt.show()
```



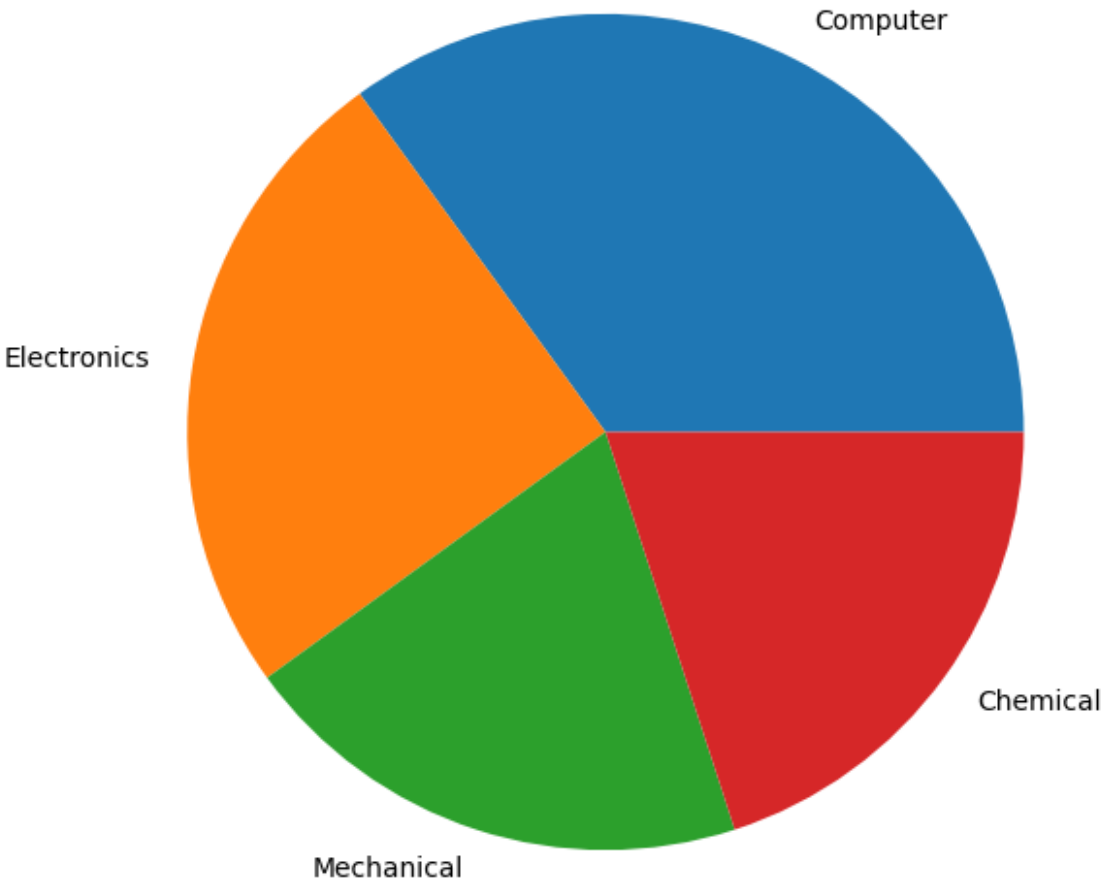
In [32]:
```
x9 = np.arange(0, 4, 0.2)

y9 = np.exp(-x9)

e1 = 0.1 * np.abs(np.random.randn(len(y9)))

plt.errorbar(x9, y9, yerr = e1, fmt = '.-')

plt.show()
```



In [33]:
```
A = [15., 30., 45., 22.]

B = [15., 25., 50., 20.]

z2 = range(4)

plt.bar(z2, A, color = 'b')
plt.bar(z2, B, color = 'r', bottom = A)

plt.show()
```



In [34]:
```
plt.figure(figsize=(7,7))

x10 = [35, 25, 20, 20]

labels = ['Computer', 'Electronics', 'Mechanical', 'Chemical']

plt.pie(x10, labels=labels);

plt.show()
```



In [ ]: